

# Trabajo Práctico N° 1: Ingeniería de Prompts y Git/GitHub

**Asignatura:** Organización Empresarial

## Estructura de Presentación

Para cada consigna se debe incluir el prompt utilizado y la captura de pantalla con la respuesta del modelo.

---

### Actividad 1: Calibración del Modelo (Temperatura)

#### 1.a. Alta Precisión (Baja Temperatura: 0.0)

**Configuración recomendada:** Ajustar la perilla de Temperatura en 0.0

**Prompt:**

Simula un parámetro de Temperatura = 0.0. Responde a la siguiente consulta de forma estrictamente técnica, objetiva, directa y sin creatividad: ¿Qué es una variable en programación?

Una **variable** en programación es un identificador asociado a un espacio de almacenamiento que contiene un valor. Dicho valor puede ser modificado durante la ejecución del programa.

#### 1.b. Alta Creatividad (Alta Temperatura: 1.0)

**Configuración recomendada:** Ajustar la perilla de Temperatura en 1.0

**Prompt:**

Actúa con un nivel de variabilidad equivalente a temperatura 1.0. Explica qué es una variable en programación inventando una historia corta y creativa con un personaje de videojuegos.

Imagínate a Mario entrando a un nivel con una mochila mágica.

En la mochila tiene una etiqueta llamada `monedas`:

```
</> Python
```

```
monedas = 10
```



Ejecutar

La variable `monedas` guarda el valor 10. Cuando Mario encuentra 5 monedas más:

```
</> Python
```

```
monedas = 15
```



Ejecutar

La etiqueta sigue siendo la misma, pero su valor cambió.

Así funciona una **variable en programación**: es un nombre asociado a un dato almacenado que puede cambiar durante la ejecución del programa.

---

## Actividad 2: Restricción de Longitud (Max Length)

**Configuración recomendada:** Ajustar Max Tokens o indicarlo explícitamente en el prompt.

**Prompt:**

Explicame qué es un condicional "if" en programación en un máximo de 10 palabras.

Un `if` ejecuta código si se cumple una condición.

---

## Actividad 3: Secuencia de Parada (Stop Sequence)

**Configuración recomendada:** Definir el delimitador de parada (por ejemplo: `###`).

**Prompt:**

Menciona 5 lenguajes de programación, escribiendo uno por línea y numerados del 1 al 5.

Detén la generación de texto inmediatamente antes de escribir el número 4. STOP

SEQUENCES: "4"

1. Python
2. JavaScript
3. C++

###

1. Python

2. Java

3. C++

---

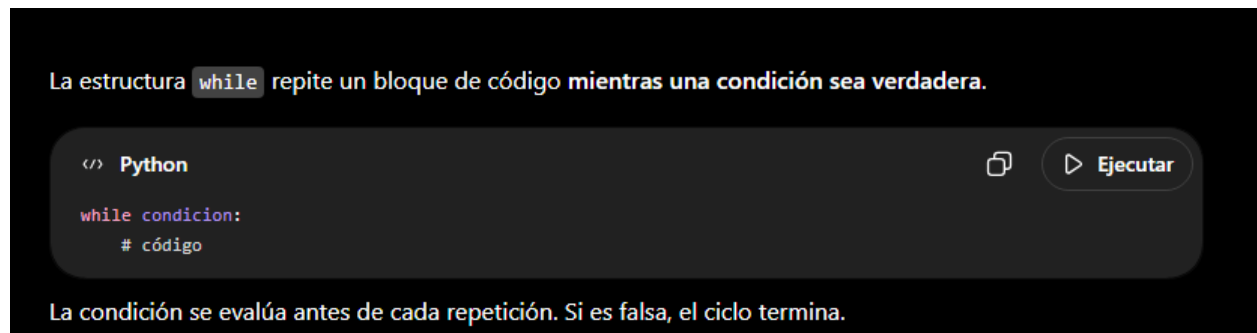
## Actividad 4: Control de Repetición (Penalizaciones)

### 4.a. Frequency Penalty (Penalización de Frecuencia)

**Configuración recomendada:** Frequency Penalty = 1.5

**Prompt:**

Explica cómo funciona la estructura "while" en programación. FREQUENCY PENALTY: 1.5



La estructura `while` repite un bloque de código mientras una condición sea verdadera.

```
</> Python
while condicion:
    # código
```

Ejecutar

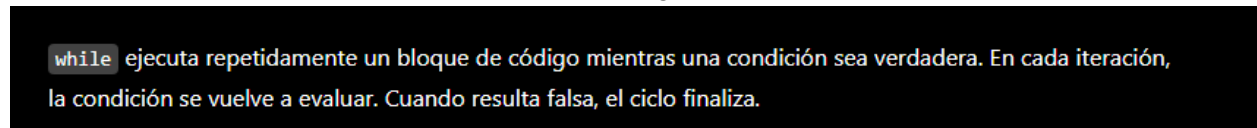
La condición se evalúa antes de cada repetición. Si es falsa, el ciclo termina.

### 4.b. Presence Penalty (Penalización de Presencia)

**Configuración recomendada:** Presence Penalty = 1.5

**Prompt:**

Explica cómo funciona la estructura "while" en programación. PRESENCE PENALTY: 1.5



`while` ejecuta repetidamente un bloque de código mientras una condición sea verdadera. En cada iteración, la condición se vuelve a evaluar. Cuando resulta falsa, el ciclo finaliza.

---

## Actividad 5: Reglas de Sintaxis Avanzada

**Configuración:** Aplicación simultánea de Prioridad Posicional, Aislamiento de Variable (delimitadores ```) y Especificidad de formato.

**Prompt:**

INSTRUCCIÓN PRINCIPAL: Tu tarea es analizar los datos de programación delimitados a continuación y clasificarlos según su tipo. Presenta la salida únicamente con el formato indicado.

```

Datos a analizar:

- "Hola Mundo"

- 25

- True

```

FORMATO DE SALIDA REQUERIDO:

- Dato: [Valor] | Tipo: [Texto / Número / Booleano]

Plaintext



- Dato: "Hola Mundo" | Tipo: Texto
- Dato: 25 | Tipo: Número
- Dato: True | Tipo: Booleano